Lab 2 – Testing and refactoring AI code

Goal - Reinforce AI output with human-driven validation.

Challenge Project: E-commerce Price Calculator with Dynamic Discounts

Scenario: Create a pricing engine that handles multiple discount types, tax calculations, and promotional rules.

AI Prompting phase:

PricingEngine class with methods:

- calculateItemPrice(item, quantity, customerTier)
- applyDiscounts(cart, promotionalCodes)
- calculateTax(subtotal, location, itemCategories)
- processBulkDiscount(items, quantity)
- validatePromotionalCode(code, customerHistory)

Human Testing & Validation Phase:

1. Generate AI Test Cases: Ask AI to create unit tests for the pricing engine
2. Run Tests: Execute the AI-generated tests and observe results
3. Identify Gaps: Look for missing test scenarios:
    - Negative quantities or prices
    - Invalid promotional codes
    - Tax calculation edge cases (tax-exempt items)
    - Discount stacking scenarios
    - Currency precision issues
    - Concurrent discount applications

Refactoring Exercise:

1. Identify code smells in AI-generated implementation
2. Refactor for better separation of concerns
3. Add missing error handling
4. Implement proper logging without exposing sensitive data
5. Add performance optimizations

© 2025 QA Limited or its affiliates. All rights reserved.

Success Metrics:

- Test coverage >90% with meaningful tests
- All edge cases identified and handled
- Performance benchmarks met
- Code passes static analysis tools

Output: use the code review checklist (See next page). Annotate the code and prioritize issues

AI-Assisted Code Review Checklist

Correctness & Functionality

- ☐ Does the code compile/run without errors?
- ☐ Are all requirements or prompt instructions implemented?
- ☐ Are inputs validated and edge cases handled?
- ☐ Are assumptions explicit, or is the AI inferring behaviour without clear justification?
- ☐ Does the code produce the expected output in sample test cases?

Readability & Maintainability

- ☐ Is the code clear and self-explanatory?
- ☐ Are names (variables, functions, classes) meaningful and consistent?
- ☐ Is there unnecessary complexity or redundant logic (over-engineering)?
- ☐ Are comments concise, accurate, and useful?
- ☐ Does the style match the project's conventions (linting/formatting)?

© 2025 QA Limited or its affiliates. All rights reserved.

- ☐ Could another developer easily understand and modify this code later?

Test Coverage

- ☐ Are there unit or integration tests for key functionality?
- ☐ Do tests check both success and failure paths?
- ☐ Are tests deterministic (no random, flaky behaviour)?
- ☐ Do coverage reports show adequate coverage (>80%)?
- ☐ Is test data realistic and not hard-coded in insecure ways?

Security

- ☐ Are all inputs validated and sanitized (especially user input)?
- ☐ Does the code avoid insecure functions or libraries?
- ☐ Are secrets, tokens, or credentials handled securely (not hard-coded or logged)?
- ☐ Does it correctly handle authentication and authorization where relevant?
- ☐ Are external calls (APIs, DB queries, file I/O) done safely?
- ☐ Have dependencies been scanned for known vulnerabilities?

Performance & Efficiency

- ☐ Is the algorithm appropriate for expected data volumes?
- ☐ Are there unnecessary loops, nested calls, or inefficient operations?
- ☐ Are external API or database calls minimized or batched when possible?
- ☐ Is memory or resource management handled properly?

Reliability & Robustness

- ☐ Does the code gracefully handle exceptions and unexpected states?
- ☐ Are fallback or recovery mechanisms implemented where needed?
- ☐ Are logging and error messages clear and actionable (no sensitive data in logs)?

Ethical & Legal Considerations

- ☐ Does the code include or reproduce any licensed or copyrighted material?
- ☐ Does it reference or copy large sections from training data examples?
- ☐ Is the AI-generated content clearly attributed or traceable if needed?

© 2025 QA Limited or its affiliates. All rights reserved.

- ☐ Does the code comply with organizational or legal AI-use policies?

Documentation & Context

- ☐ Is there a clear explanation of what the code does and how to use it?
- ☐ Are AI-generated sections labeled or noted in comments/commits?
- ☐ Is rationale provided for non-obvious design choices?
- ☐ Are dependencies and setup steps documented?

© 2025 QA Limited or its affiliates. All rights reserved.